HW5: Online Store, Product API

Part II: Product API

Go + Gin service implementing the Product endpoints from api.yaml. Products are stored in an in memory map keyed by product ID, guarded by a sync.RWMutex for safe concurrent access.

Where things are

- Server code: src/product-service.go
- Dockerfile: src/Dockerfile
- Infrastructure: terraform/ (Part III)

Run locally

```
cd src
go run .
# server listens on :8080
```

Run in Docker

```
cd src
docker build -t product-api .
docker run -d -p 8080:8080 product-api
```

Endpoints and every response code

GET /products/{productId} - 200 product found, returns the Product JSON - 404 product not found, or productId not a positive integer - 500 internal server error (Gin recovery on panic)

POST /products/{productId}/details - 204 product details stored successfully - 400 invalid input (bad path id or body failing schema validation) - 404 product not found (applies if the product must pre exist; this design upserts) - 500 internal server error

Example calls and responses

404, GET a product that does not exist: curl -i http://localhost:8080/products/1

400, POST an invalid body: curl -i -X POST http://localhost:8080/products/1/details -H “Content-Type: application/json” -d ‘{“sku”:““}’

204, POST a valid product: curl -i -X POST http://localhost:8080/products/1/details -H “Content-Type: application/json”
-d ‘{“product_id”:1,“sku”:“ABC-123-XYZ”,“manufacturer”:“Acme Corporation”,“category_id”:456,“weight”:1250,“some_other_id”:789}’

200, GET the product just added: curl -i http://localhost:8080/products/1

400, POST with a non integer path id: curl -i -X POST http://localhost:8080/products/abc/details -H “Content-Type: application/json” -d ‘{}’

Part V: Observability

RED signals

The server exposes /metrics (Prometheus) and emits one JSON log line per request. - Rate: sum(rate(http_requests_total[1m])) - Errors: sum(rate(http_errors_total[1m])) / sum(rate(http_requests_total[1m])) - Duration p95: histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[1m])) by (le))

Prometheus and Grafana run locally via observability/docker-compose.yml. Prometheus scrapes the server’s /metrics every 5s; the Grafana dashboard “Product API RED” has one panel per signal. See the dashboard screenshot showing idle vs loaded behaviour during a Locust run.

Sample request logs (from the loaded run)

{“time”:“...”,“level”:“INFO”,“msg”:“request”,“method”:“GET”,“endpoint”:“/products/:productId”,“status”:200
{“time”:“...”,“level”:“INFO”,“msg”:“request”,“method”:“POST”,“endpoint”:“/products/:productId/details”,“st
{“time”:“...”,“level”:“INFO”,“msg”:“request”,“method”:“GET”,“endpoint”:“/products/:productId”,“status”:404

Dashboard vs logs

I look at the Grafana dashboard for live, aggregate health, spotting a rate drop or latency spike as it happens. I look at the logs for after-the-fact, per-request debugging, finding which specific endpoint and status caused a problem when nobody had Grafana open at the time.